

Caminos que olvidan los ciclos

Una biyección verificada por máquina entre los caminos tipo árbol de un grafo y los caminos del árbol de caminos de Godsil, en Lean 4

Carles Marín*

Investigador independiente

karlesmarin@gmail.com

Junio de 2026

Resumen

Esta es la tercera pieza de una serie que construye, en Lean 4 / Mathlib, los dos lados de la fórmula de la traza de emparejamientos/Ihara. En *Desplegar un grafo en un árbol* [13] usé el *árbol de caminos* $T(G, v)$ de Godsil para demostrar que el polinomio de emparejamientos de un grafo tiene todas sus raíces reales: el árbol de caminos es un bosque, y un bosque porta un espectro simétrico. Aquí pongo el *mismo bosque* a un segundo uso. Un camino cerrado de G es *tipo árbol* cuando nunca cierra a sabiendas un ciclo—en cada paso o avanza a un vértice nuevo o retrocede a su padre, exactamente como si caminara sobre un árbol. Doy una demostración verificada por máquina, **sorry**-libre, de que los caminos cerrados tipo árbol de G con base en v están en biyección, que preserva la longitud, con los caminos cerrados de $T(G, v)$ con base en su raíz, realizada por la proyección hacia abajo $\pi: T(G, v) \rightarrow G$ que envía un camino a su extremo. Como corolario, el conteo de caminos tipo árbol se vuelve una suma de potencias de la matriz de adyacencia de los árboles de caminos,

$$\text{treeLikeWalkCount}(G, k) = \sum_v [A(T(G, v))^k]_{\text{raíz}, \text{raíz}},$$

y, como cada $T(G, v)$ es un bosque, su polinomio de emparejamientos es igual al polinomio característico de su matriz de adyacencia. No he hallado una formalización previa del árbol de caminos de Godsil como objeto que cuenta caminos, ni de la biyección resultante, ni del predicado intrínseco “tipo árbol”, en ningún asistente de demostración. Soy exacto sobre el límite: este artículo construye la mitad *combinatoria* del teorema de los momentos de Godsil $p_k = \sum_i \theta_i^k = \text{treeLikeWalkCount}(G, k)$. La mitad *espectral* restante—la identidad del resolvente para una entrada y el paso de Newton univariado que convierten $[A(T)^k]_{\text{raíz}}$ en un coeficiente de $\mu(G - v)/\mu(G)$ —queda trazada en la Sección 7, no construida. Nada de esto es matemática nueva; la contribución es la formalización, y el hecho de que el árbol de caminos ahora cuenta caminos en la misma biblioteca donde ya diagonaliza un espectro.

Mapa para el lector. Si lees una sola cosa, lee el Teorema 1—la biyección—y la razón de una línea por la que se cumple: el árbol de caminos es el grafo *desenrollado* hasta que los caminos se autointersecan, una *inmersión* de G en la que un camino se levanta a lo sumo de una forma, y se levanta exactamente cuando es tipo árbol. Todo lo demás es la versión cuidadosa de esa frase: un lema de inyectividad local (Sección 4), un invariante `liftSeq`↔camino que hace “tipo árbol” computable y lo casa con el levantamiento (Sección 5), y el levantamiento mismo (Sección 6). El límite honesto—la mitad espectral está trazada, no construida—se enuncia tan llanamente como el resultado.

*Formalizado con asistencia de IA (Claude, Anthropic); la matemática y todas las afirmaciones son responsabilidad del autor. La metodología se discute en la Sección 8.

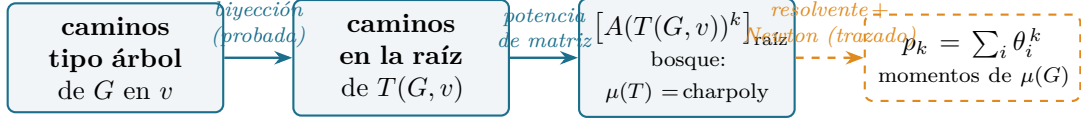


Figura 1: El recorrido del artículo, y dónde se detiene. El conteo de caminos tipo árbol de G iguala al conteo de caminos en la raíz del árbol de caminos de Godsil (la biyección, Teorema 1), que es una potencia de la matriz de adyacencia del árbol (Teorema 2); y cada árbol de caminos es un bosque, así que el polinomio característico de esa matriz es el polinomio de emparejamientos (Teorema 3). Las flechas sólidas están verificadas por máquina aquí; la flecha discontinua hacia los momentos p_k —el resolvente de una entrada, la identidad del cociente de Godsil y el paso de Newton univariado—está trazada en la Sección 7, no construida.

1. Introducción: un árbol, dos cosechas

¿Cuándo un camino sobre un grafo no se da cuenta de que el grafo tiene ciclos—y por qué contar esos caminos olvidadizos debería recuperar los momentos de un polinomio que no es el polinomio característico de ninguna matriz? Las dos mitades de esa pregunta resultan tener una sola respuesta, un árbol; este artículo verifica por máquina la mitad combinatoria de ella.

Responderla—y explicar por qué un solo artificio combinatorio zanja dos hechos sobre un grafo que parecen no relacionados—pide que se encuentren tres hilos de historia.

Un polinomio con raíces reales y sin matriz (1972). Un grafo finito G tiene un polinomio de emparejamientos $\mu(G, x) = \sum_k (-1)^k m_k(G) x^{n-2k}$, donde $m_k(G)$ cuenta sus emparejamientos de k aristas [2, 3]. En 1972 Heilmann y Lieb, estudiando sistemas de monómeros y dímeros en mecánica estadística, demostraron que sus raíces son siempre reales [1]—un enunciado sobre transiciones de fase en física, espectral en combinatoria. Pero las raíces reales son la firma de una *matriz simétrica*, y $\mu(G)$ no es el polinomio característico de ninguna matriz fija salvo que G sea un bosque. Así que la pregunta se afila: si no hay matriz, ¿de dónde salen las raíces reales, y el contenido combinatorio de sus sumas de potencias $p_k = \sum_i \theta_i^k$?

Un árbol, dos teoremas (1981). La respuesta de Godsil fue un solo artificio [4, 5, 6]. Dado un vértice v , el *árbol de caminos* $T(G, v)$ tiene como vértices los caminos simples de G que empiezan en v , y une un camino a otro cuando el segundo extiende al primero por una sola arista. Es un bosque—la truncación finita del *recubrimiento universal* de G desenrollado desde v —y de él Godsil sacó dos conclusiones. Primero, $\mu(G) \mid \mu(T(G, v))$, y el polinomio de emparejamientos de un bosque es el polinomio característico de su matriz de adyacencia, luego de raíces reales; esto recupera Heilmann–Lieb, y es lo que formalizó la Parte II de esta serie [13]. Segundo—el tema de este artículo—el árbol de caminos *cuenta caminos*.

Caminos, recubrimientos y una fórmula de traza. El recubrimiento universal es el objeto más profundo tras el árbol finito: el radio espectral del árbol infinito $(\Delta - 1)$ -regular es $2\sqrt{\Delta - 1}$, el borde *Ramanujan*, que es también la cota de Godsil sobre las raíces de emparejamiento, y el espectro del recubrimiento está gobernado por el polinomio de emparejamientos [7, 8]. Del lado finito esto es una *fórmula de traza*: $\text{tr}(A^k)$ cuenta *todos* los caminos cerrados de G , mientras que p_k cuenta solo los *tipo árbol*; por debajo de la cintura todo camino es tipo árbol, así que ambos coinciden, y la primera diferencia cuenta los ciclos más cortos. El lado ciclo, $\text{tr}(A^k)$ y su refinamiento

de no-retroceso, es el complemento *Bass* de esta serie [14, 9, 10]; este artículo construye la mitad combinatoria del lado árbol.

Un camino cerrado de G es *tipo árbol* cuando, visto paso a paso, solo avanza a un vértice nuevo o retrocede al vértice del que vino: nunca atraviesa una arista de vuelta a un vértice anterior que no sea su padre, así que nunca cierra a sabiendas un ciclo. (Puede, en conjunto, atravesar las tres aristas de un triángulo—siempre que cada *camino* intermedio siga simple; es justo ahí donde falla la definición ingenua, Figura 2.) La observación de Godsil es que estos son precisamente los caminos que G hereda de $T(G, v)$: un camino cerrado en la raíz del árbol de caminos, empujado hacia abajo a G registrando solo el extremo actual, es un camino cerrado tipo árbol en v , y todo camino tipo árbol surge así, de forma única (Figura 3). Sumando sobre v y sobre los caminos de longitud k se obtiene `treeLikeWalkCount`(G, k), y el teorema de los momentos lo identifica con $p_k(G)$.

Por qué formalizarlo. El polinomio de emparejamientos está en la intersección de tres hilos de esta serie—raíces reales (Partes I–II [12, 13]), la fórmula de la traza de emparejamientos/Ihara (el complemento *Bass* [14]) y, por el borde de banda $2\sqrt{\Delta - 1}$, los grafos de Ramanujan [11]. La fórmula de la traza tiene un lado *ciclo*, $\text{tr}(A^k)$ contando todos los caminos cerrados, y un lado *árbol*, p_k contando los tipo árbol; por debajo de la cintura coinciden, y la primera diferencia es un conteo de ciclos cortos. Para enunciar ese puente dentro de un asistente de demostración hace falta el lado árbol como objeto honesto, y el lado árbol es precisamente un enunciado sobre el árbol de caminos. La Parte II construyó el árbol de caminos por su espectro; este artículo enseña al mismo objeto de Lean a contar.

Qué hay de nuevo aquí, y qué no. La matemática es de Godsil. La contribución es la construcción verificada por máquina: el árbol de caminos como grafo que cuenta caminos, la proyección hacia abajo como homomorfismo de grafos que es una *inmersión*, un predicado decidible intrínseco `IsTreeLike` que computa el levantamiento como una disciplina de pila, el invariante que ata esa pila al vértice actual del árbol de caminos, el levantamiento en la dirección sobreyectiva, y la biyección de cardinales resultante. De ninguno de ellos hallé una formalización previa en ningún asistente de demostración. El enunciado principal depende solo de los tres axiomas estándar (`propext`, `Classical.choice`, `Quot.sound`); no hay sorry.

2. Los dos lados, mantenidos aparte

¿Qué, exactamente, estamos casando—y cómo evitamos que los dos lados sean el mismo objeto con dos sombreros? Una biyección solo dice algo cuando sus extremos se definen sin referencia uno al otro; así que fijo ambos, y solo después dejo que se encuentren. En todo lo que sigue, G es un `SimpleGraph` sobre un tipo de vértices V , y $v \in V$ es un punto base. Reutilizo el árbol de caminos de la Parte II tal cual.

Definición 1 (Árbol de caminos, `pathTree`). *Los vértices de $T(G, v) = \text{pathTree } G \ v$ son los caminos simples de G que empiezan en v ($\text{PathFrom } v = \sum_u G.\text{Path } v \ u$). Dos son adyacentes cuando uno se obtiene del otro añadiendo una sola arista al final (*Grows*). La raíz es el camino trivial en v (pathTreeRoot). Es un bosque: `pathTree_isAcyclic`.*

Definición 2 (Proyección hacia abajo, `pathTreeProj`). $\pi: T(G, v) \rightarrow G$ envía un camino a su extremo, $\pi\langle u, p \rangle = u$. La adyacencia en el árbol de caminos significa que un camino extiende al otro por una arista, así que los dos extremos son adyacentes en G ; por tanto π es un homomorfismo de grafos, y $\pi(\text{raíz}) = v$.

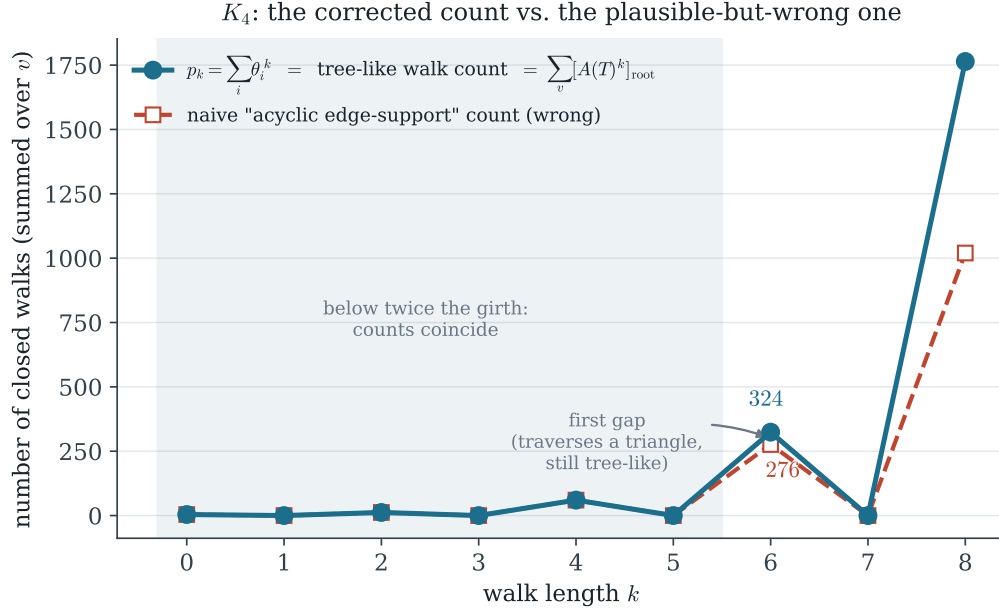


Figura 2: Por qué importa la definición, en K_4 . Las sumas de potencias de emparejamiento p_k (turquesa) coinciden con el conteo tipo árbol / del árbol de caminos para todo k —la biyección de este artículo hace exacta la segunda igualdad. El conteo plausible de “soporte de aristas acíclico” (rojo) concuerda hasta $k = 5$ pero luego subcuenta, 276 frente a 324 en $k = 6$, porque un camino tipo árbol puede atravesar las tres aristas de un triángulo mientras cada camino intermedio sigue simple. Una comprobación numérica barata atrapó la definición equivocada antes de gastar esfuerzo formal en ella.

El predicado tipo árbol se define de forma intrínseca, sin referencia al árbol de caminos, para que los dos lados de la biyección sean objetos genuinamente distintos por casar. Un camino cerrado w en v se alimenta, vértice a vértice (su soporte salvo la primera entrada), a un *levantamiento* determinista `liftSeq` que enhebra una pila—el camino simple actual desde v . Para cada vértice siguiente c : si c no está en la pila, lo *apila* (un AVANCE); si c es el penúltimo vértice de la pila, *desapila* (un RETROCESO al padre); en otro caso el levantamiento *falla*. Los dos casos son mutuamente excluyentes, así que el levantamiento es un *fold*.

Definición 3 (Tipo árbol, `Walk.IsTreeLike`). w es tipo árbol si su levantamiento, comenzado desde la pila raíz $[v]$, vuelve a $[v]$: `liftSeq w.support.tail [v] = some [v]`. Es decidible.

Esta definición es la corregida. La elección ingenua—“el soporte de aristas de w es acíclico”—es *falsa* para el teorema de los momentos: en K_4 con $k = 6$ subcuenta (276 frente a $p_6 = 324$), porque un camino tipo árbol puede atravesar las aristas de un ciclo en conjunto. El predicado `liftSeq` se validó numéricamente contra las sumas de potencias de emparejamiento en P_4, C_4, C_5, K_4 y los grafos de Petersen y $K_{3,3}$ para $k \leq 8$ antes de formalizarlo (Figura 2).

3. El resultado

Teorema 1 (La biyección, `card_treeLike_eq_pathTreeWalks`). Para todo grafo finito G , punto base v y longitud k , la aplicación $W \mapsto W.\text{map } \pi$ es una biyección de los caminos cerrados de

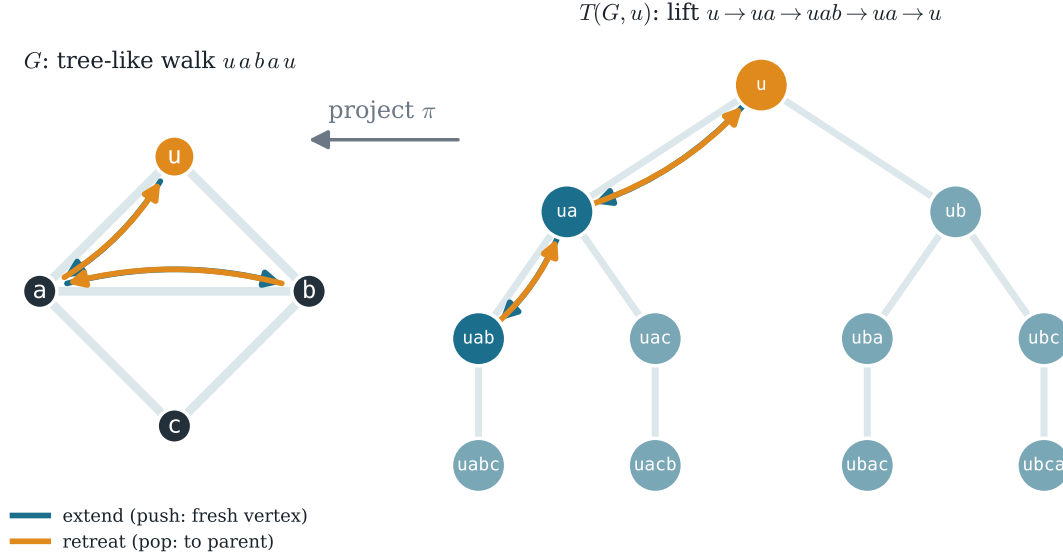


Figura 3: La biyección en el diamante $G = K_4 - e$, raíz u . Un camino cerrado en la raíz del árbol de caminos $T(G, u)$ (derecha), $u \rightarrow ua \rightarrow uab \rightarrow ua \rightarrow u$, se proyecta bajo π (un camino $\mapsto$ su extremo) a un camino cerrado tipo árbol de G (izquierda), $uabau$. Cada paso es un AVANCE (turquesa, ir a un vértice nuevo / apilar) o un RETROCESO (ámbar, volver al padre / desapilar). El camino de la izquierda recorre las aristas ua, ab hacia afuera y las retraza de vuelta; el levantamiento de la derecha hace explícita la historia del camino, y es el único camino sobre ella.

longitud k en la raíz de $T(G, v)$ sobre los caminos cerrados tipo árbol de longitud k de G en v . En particular sus cardinales coinciden:

$$\#\{\text{caminos tipo árbol de } G \text{ en } v, \text{ longitud } k\} = \#\{\text{caminos en la raíz de } T(G, v), \text{ longitud } k\}.$$

¿Qué hace falta para fiarse de una biyección? Una aplicación, y su promesa cumplida en dos direcciones. La aplicación es π leída sobre caminos enteros, $W \mapsto W.\text{map } \pi$; su promesa son tres hechos, y las tres secciones de demostración que siguen son, en orden, exactamente esos tres. La aplicación debe *aterrizar donde debe*: un camino-árbol proyectado es siempre tipo árbol, así que π lleva los caminos del árbol *dentro* de los tipo árbol de G (Lema 1, ganado en la Sección 5). Debe *no olvidar nada*: dos caminos-árbol no proyectan la misma sombra (Lema 2, Sección 4). Y debe *no perder nada*: todo camino tipo árbol es la sombra de algún camino—el levantamiento existe (Lema 3, Sección 6). Aterrizas, inyectiva, sobreyectiva; eso es todo el Teorema 1, y el resto del artículo son esas tres palabras hechas precisas.

Lema 1 (La imagen es tipo árbol, `pathTreeProj_map_isTreeLike`). *La π -proyección de cualquier camino cerrado en la raíz de $T(G, v)$ es tipo árbol.*

Lema 2 (Inyectividad, `pathTreeProj_walk_injective`). *$W \mapsto W.\text{map } \pi$ es inyectiva sobre los caminos de $T(G, v)$.*

Lema 3 (Sobreyectividad / el levantamiento, `exists_root_lift`). *Todo camino cerrado tipo árbol de G en v es $W.\text{map } \pi$ para algún camino cerrado W en la raíz de $T(G, v)$.*

Una biyección que solo renombra un conjunto finito es contabilidad—¿qué compra entonces esta? Su valor es que los dos lados son *tipos* de objeto distintos: a la izquierda, un conteo combinatorio; a la derecha, caminos sobre un *árbol*. Y un árbol tiene una matriz. Así que el conteo de la derecha es una potencia de matriz, una potencia de matriz es una cantidad espectral, y el renombrado nos ha movido, calladamente, al lado espectral, donde vive el resto del teorema de los momentos. Dos corolarios dan el paso.

Teorema 2 (Forma espectral, `treeLikeWalkCount_eq_sum_pathTree_adjMatrix_pow`).

$$\text{treeLikeWalkCount}(G, k) = \sum_{v \in V} [A(T(G, v))^k]_{\text{raíz}, \text{raíz}}.$$

Teorema 3 (Puente del bosque, `matchingPoly_pathTree_eq_charpoly`). Cada árbol de caminos es un bosque, así que su polinomio de emparejamientos es el polinomio característico de su matriz de adyacencia: $\mu(T(G, v)) = \text{charpoly}(A(T(G, v)))$.

4. El árbol de caminos es una inmersión

Si el árbol de caminos es el grafo desenrollado, ¿qué impide que un camino se desenrolle de forma ambigua—que tenga dos levantamientos distintos? La respuesta es local, y es todo el motor de la inyectividad. Los vecinos de un vértice p del árbol de caminos son su único padre (quitar la última arista) y sus hijos (añadir una arista cada uno). La proyección π envía el padre al penúltimo vértice de p , que está sobre p , y cada hijo a su extremo nuevo, que no lo está; y hijos distintos tienen extremos distintos. Así que π separa los vecinos de cada vértice: `pathTreeProj_injOn_neighborSet`. La mitad del padre es `Grows.parent_unique` de la Parte II; su dual `Grows.child_unique` (una extensión por una arista queda determinada por su extremo) es nuevo aquí.

Esta inyectividad local es exactamente la propiedad de inmersión: π es localmente inyectiva pero no localmente sobreyectiva—un G -vecino del extremo de p que ya está sobre p no tiene preimagen por encima de p . Esa asimetría es toda la historia. Y es también una vieja con otro traje: una aplicación recubridora, en el sentido de la topología de Poincaré, levanta *todo* camino de forma única; una inmersión solo levanta los caminos que nunca se doblan sobre sí mismos—y “nunca se dobla hacia un vértice que no es el padre” es precisamente lo que significa tipo árbol. Un camino en $T(G, v)$ queda determinado, paso a paso, por su π -imagen junto con su inicio, porque en cada vértice el siguiente vértice del árbol de caminos está forzado por la inyección local; una inducción que imita a `Walk.map_injective_of_injective` de Mathlib, con la hipótesis global reemplazada por la local, da el Lema 2.

5. El invariante `liftSeq`↔camino

IsTreeLike es una disciplina de pila sobre G solo—el árbol de caminos no aparece en él—¿por qué debería detectar exactamente los caminos que vienen del árbol? Porque la pila es el vértice del árbol disfrazado. Leer un camino como una sucesión de apilados y desapilados es el movimiento más antiguo de la combinatoria enumerativa—el problema de la votación, las palabras equilibradas de von Dyck, los números de Catalan viven todos sobre la misma pila—y aquí es el puente, en forma de un solo invariante: al caminar $T(G, v)$, ejecutar `liftSeq` sobre el camino proyectado rastrea el vértice actual del árbol de caminos.

Lema 4 (Un paso, `liftSeq_step`). Para una arista $s \sim y$ del árbol de caminos y cualquier cola M , `liftSeq(y1 :: M)` `s.support = liftSeq M` y `y.support`: un paso a un hijo es un APILAR, un paso al padre es un DESAPILAR.

On $p = \langle uab \rangle$: π separates the neighbours of b
(an immersion, not a covering)

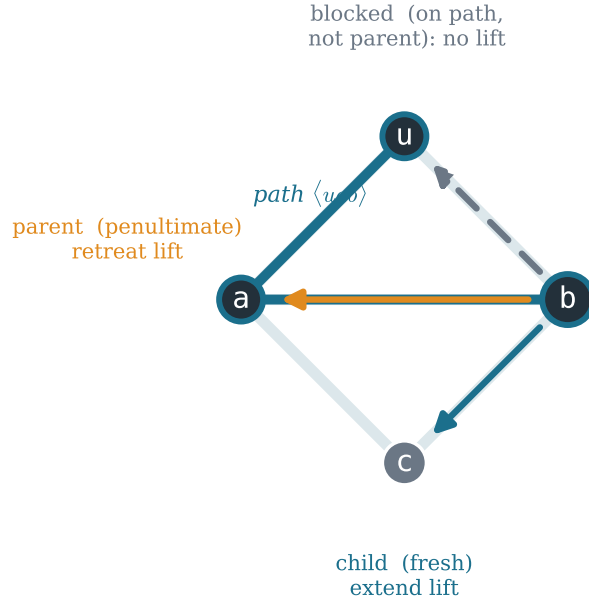


Figura 4: La inmersión, en el vértice $p = \langle uab \rangle$ del árbol de caminos (el camino uab , turquesa). El paso siguiente mira los vecinos del extremo b , que en G son a, c, u . Cada uno cae en exactamente un caso: a es el penúltimo de p (el *padre*, un retroceso), c es nuevo (un *hijo*, un avance), y u está sobre p pero no es su padre—*bloqueado*, sin levantamiento por encima de p . Así π es inyectiva sobre los vecinos de p pero no sobreyectiva; la dirección bloqueada es por lo que el árbol de caminos es una inmersión, y por lo que un camino proyectado solo puede ser tipo árbol.

Lema 5 (Invariante, `liftSeq_map_invariant`). *Para un camino $W: s \rightarrow x$ del árbol de caminos, $\text{liftSeq}(W.\text{map } \pi).\text{support.tail } s.\text{support} = \text{some } x.\text{support}$.*

El Lema 1 es el caso $s = x = \text{raíz}$, cuyo soporte de camino es $[v]$. El mismo invariante impulsa el levantamiento: es la contabilidad que, ejecutada al revés, reconstruye el camino del árbol de caminos a partir de un camino tipo árbol.

6. El levantamiento: trepar de vuelta al árbol

Todo camino tipo árbol debería venir del árbol—pero ¿cómo exhibimos el camino del que viene, cuando sus propios extremos en el árbol no se dan sino que se computan? La sobreyectividad es la única dirección que construye un camino en el tipo dependiente $(T(G, v)).\text{Walk}$, y sus extremos se computan, no se dan. La construcción (`exists_lift`) es una recursión sobre el G -camino: la hipótesis “`liftSeq` acepta w desde el vértice actual” suministra, en cada paso, la decisión AVANCE/RETROCESO, así que no hay rama de fallo. El vértice actual del árbol de caminos se lleva como la terna explícita $\langle a, pp, hpp \rangle$ (extremo, camino, prueba-de-que-es-camino), lo que esquiva las igualdades heterogéneas que un acumulador de tipo Σ forzaría. Dos fricciones más se manejan con el mismo truco: la recursión demuestra igualdad de *soportes* (una `List V`, libre de tipos dependientes) en lugar de igualdad de

IsTreeLike: the walk $uabau$ as a stack discipline (stack at step i = path-tree vertex)

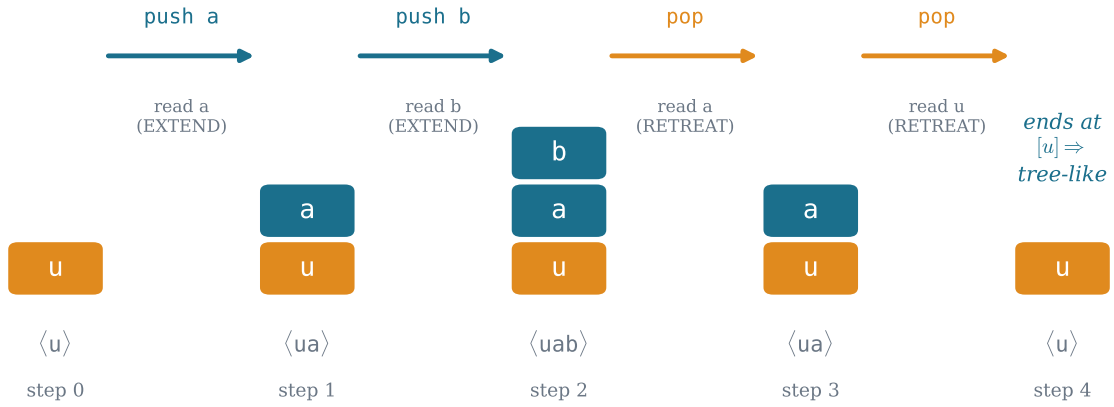


Figura 5: IsTreeLike como disciplina de pila—el predicado `liftSeq` ejecutado sobre el camino $uabau$. La pila es el camino simple actual desde u ; cada vértice siguiente se lee y o se APILA (avance, turquesa: un vértice nuevo) o se DESAPILA (retroceso, ámbar: es el penúltimo). El camino es tipo árbol si la pila termina de vuelta en $[u]$. La pila en el paso i es exactamente el vértice del árbol de caminos sobre el que se sitúa el levantamiento— $\langle u \rangle, \langle ua \rangle, \langle uab \rangle, \langle ua \rangle, \langle u \rangle$ —que es el contenido del invariante, Lema 5.

caminos, y `Walk.support_injective` eleva la igualdad de soportes a igualdad de caminos una vez fijados los extremos en el corolario del camino cerrado (Lema 3). El levantamiento vuelve a la raíz porque su soporte de camino final es $[v]$, forzando que el vértice-árbol final sea el camino trivial.

La biyección (Teorema 1) es entonces `Finset.card_bij` sobre $W \mapsto W.\text{map } \pi$, con el Lema 1 para la buena definición, el Lema 2 para la inyectividad y el Lema 3 para la sobreyectividad; las longitudes se preservan porque `map` las preserva.

7. El límite: qué se construye y qué se traza

La biyección cuenta; el teorema de los momentos iguala ese conteo con un espectro. ¿Dónde, exactamente, termina el terreno verificado por máquina y empieza el álgebra trazada-pero-sin-construir? El teorema de los momentos de Godsil es

$$p_k(G) = \sum_i \theta_i^k = \text{treeLikeWalkCount}(G, k).$$

Este artículo construye su mitad *combinatoria*—todo hasta la forma espectral (Teorema 2) y el puente del bosque (Teorema 3) inclusive. La mitad *espectral* se traza aquí pero no se formaliza; la enuncio llanamente para que el lector sepa exactamente dónde acaba el terreno verificado por máquina.

La cadena restante es álgebra estándar. Para cada árbol de caminos $T = T(G, v)$, el resolvente de la entrada raíz-raíz genera los conteos de caminos,

$$\sum_{k \geq 0} [A(T)^k]_{\text{raíz}} X^k = \frac{\text{charpolyRev}(A(T)_{\widehat{\text{raíz}}})}{\text{charpolyRev}(A(T))},$$

el caso de una entrada de $\text{adj}(1 - XA) = \det(1 - XA) (1 - XA)^{-1}$, que esta biblioteca ya tiene en forma matricial (`smul_resolventSeries_eq_adjugate`, `coeff_resolventSeries`). Lo que *aún no* está en Mathlib, y es el único lema de álgebra lineal genuinamente nuevo que la completación necesita, es la identidad del menor principal $\text{adj}(M)_{ii} = \det(M \upharpoonright_{j \neq i})$ sobre un tipo de índices finito arbitrario (Mathlib solo tiene la expansión por cofactores para `Fin`). Concediéndolo, el puente del bosque convierte cada `charpolyRev` en un polinomio de emparejamientos invertido; la `godsil_identity` de la Parte II, $\mu(G) \mu(T - \text{raíz}) = \mu(G - v) \mu(T)$, reescribe el cociente como $\mu(G - v)/\mu(G)$; la ley de la derivada por borrado de vértice $\sum_v \mu(G - v) = X \mu'(G)$ (ya demostrada, `sum_matchingPoly_deleteIncidenceSet`) lo suma a $\mu'(G)/\mu(G)$; y la identidad de Newton univariada iguala esa derivada logarítmica con $\sum_k p_k X^k$. Igualando los coeficientes de X^k se cierra el teorema. El último paso es venerable en sí: las identidades de Newton, que relacionan las sumas de potencias de un polinomio con sus coeficientes, las escribió Girard en 1629 y Newton una generación más tarde, y el teorema de los coeficientes de Sachs de 1964 lee esos coeficientes en los propios subgrafos del grafo. Dos de los eslabones—el lema del menor principal y el puente de Newton para raíces univariadas—son resultados generales ausentes de Mathlib, y ambos se guardan en la zona `MathlibPR` del repositorio en forma canónica, sin grafos.

8. Implicaciones y metodología

El autor fijó los objetivos y dirimió la matemática; el asistente (Claude, Anthropic) redactó las demostraciones y esta prosa. Todo teorema citado aquí compila `sorry-libre` contra la revisión fijada de Mathlib [15] y depende solo de los tres axiomas estándar, lo que el lector puede reverificar con `#print axioms`. La definición “tipo árbol” corregida es en sí un pequeño dato metodológico: la lectura natural por soporte acíclico es plausible, compila, y es *falsa* para el teorema al que debe servir; una comprobación numérica barata contra las sumas de potencias de emparejamiento la atrapó antes de gastar esfuerzo formal, y la misma comprobación certifica la definición `liftSeq` que la reemplazó.

Para qué sirve—y para qué no. Los objetos de aquí no son exóticos. El polinomio de emparejamientos es la función de partición monómero–dímero de la mecánica estadística [1]; los momentos de sus raíces son la materia prima del método de los momentos para acotar autovalores; la cota que esos momentos obedecen, $2\sqrt{\Delta - 1}$, es el umbral que un grafo debe superar para ser un expansor; y el objeto espectral de al lado, el operador de no-retroceso, fija el umbral de percolación de una red [9, 14]. Este artículo no añade nada a eso. Solo pone una mitad del puente que esos métodos cruzan—el lado árbol de la fórmula de la traza—sobre terreno certificado, en la misma biblioteca que el lado ciclo, y deja tras de sí un pequeño objeto reutilizable, el árbol de caminos, que ya hizo un trabajo (un espectro real, Parte II) y ahora hace un segundo. Ningún algoritmo y ninguna red: un ciento, y uno modesto.

La matemática es de Godsil, de hace cuatro décadas. Solo la he llevado, con cuidado, a una forma que una máquina comprobará: en la Parte II el árbol de caminos porta un espectro real, y aquí cuenta los caminos que un grafo confunde con un árbol. El lado ciclo de la fórmula de la traza de emparejamientos/Ihara— $\text{tr}(A^k)$ —está en la misma biblioteca (el complemento *Bass* [14]); si algún día se construyera la mitad espectral de la Sección 7, el lado árbol p_k se encontraría con él, y la fórmula de la traza sería una sola línea verificada por máquina. Eso queda, sin disculpas, para otro día.

9. Preguntas que quedan abiertas

Una formalización es también una lista de las próximas preguntas, afiladas hasta el punto en que se le podrían plantear a una máquina. Estas son las que este artículo deja sobre la mesa.

- La completación de la Sección 7 pende de un solo lema general, ausente de Mathlib: la identidad del menor principal $\text{adj}(M)_{ii} = \det(M \upharpoonright_{j \neq i})$ sobre un tipo de índices finito arbitrario. ¿Es la ruta limpia un reindexado a `Fin` y la expansión por cofactores existente, o hay una demostración sin base que nunca nombre un orden?
- El predicado “tipo árbol” corregido es una *disciplina de pila*. ¿Es la única definición intrínseca razonable, o la lectura de reducción-libre-a-trivial (un camino que se reduce a la palabra vacía por cancelación de retrocesos) define la misma clase para todo k —y, de ser así, es la equivalencia más barata de demostrar que cualquiera de los dos lados de la biyección?
- La biyección preserva la longitud y es local en el vértice. ¿Se refina a un isomorfismo de las *funciones generatrices* antes de cualquier insumo espectral, es decir, puede identificarse $\sum_k (\#\text{tipo árbol}) X^k$ con una entrada del resolvente del árbol de caminos de forma puramente combinatoria, dejando el polinomio de emparejamientos fuera hasta el final?
- Por debajo de la cintura, los caminos tipo árbol son todos los caminos, así que el conteo iguala a $\text{tr}(A^k)$ localmente; la primera diferencia, en $k = \text{cintura}$, cuenta los ciclos más cortos. ¿Puede hacerse que la biyección del árbol de caminos *vea* esa diferencia—que produzca el término de corrección que cuenta ciclos como la obstrucción a que un camino se levante?
- El mismo bosque porta el borde de banda $2\sqrt{\Delta - 1}$ (Parte II) y, aquí, los conteos de caminos cuya tasa de crecimiento es ese mismo borde. ¿Es la cota de Ramanujan sobre las raíces de emparejamiento un *corolario* de esta biyección más una comparación con el árbol infinito $(\Delta - 1)$ -ario, formalizable sin volver a entrar en la maquinaria de familias entrelazadas?

Y una para conservar. *Cuando una máquina certifica que los caminos tipo árbol de un grafo son exactamente los caminos de un árbol, ¿ha demostrado que el grafo es, a efectos de contar, un árbol—o solo que hemos nombrado el sentido preciso en que se niega a serlo?*

Referencias

- [1] O. J. Heilmann and E. H. Lieb, *Theory of monomer–dimer systems*, Comm. Math. Phys. **25** (1972), 190–232.
- [2] E. J. Farrell, *An introduction to matching polynomials*, J. Combin. Theory Ser. B **27** (1979), 75–86.
- [3] L. Lovász and M. D. Plummer, *Matching Theory*, North-Holland Math. Stud. **121**, Amsterdam, 1986.
- [4] C. D. Godsil, *Matchings and walks in graphs*, J. Graph Theory **5** (1981), 285–297.
- [5] C. D. Godsil and I. Gutman, *On the theory of the matching polynomial*, J. Graph Theory **5** (1981), 137–144.
- [6] C. D. Godsil, *Algebraic Combinatorics*, Chapman & Hall, Nueva York, 1993.

- [7] O. Angel, J. Friedman, and S. Hoory, *The non-backtracking spectrum of the universal cover of a graph*, Trans. Amer. Math. Soc. **367** (2015), 4287–4318.
- [8] T. J. Spier, *Eigenvalues of universal covers and the matching polynomial*, preprint, 2025. [arXiv:2510.05041](https://arxiv.org/abs/2510.05041).
- [9] K. Hashimoto, *Zeta functions of finite graphs and representations of p -adic groups*, en *Automorphic Forms and Geometry of Arithmetic Varieties*, Adv. Stud. Pure Math. **15**, 1989, 211–280.
- [10] H. Bass, *The Ihara–Selberg zeta function of a tree lattice*, Internat. J. Math. **3** (1992), 717–797.
- [11] A. Marcus, D. A. Spielman, and N. Srivastava, *Interlacing families I: Bipartite Ramanujan graphs of all degrees*, Ann. of Math. **182** (2015), 307–325.
- [12] C. Marín, *Random Signs into Matchings: the Godsil–Gutman estimator and real-rootedness in Lean 4* (Parte I), 2026. Zenodo, DOI [10.5281/zenodo.20517350](https://doi.org/10.5281/zenodo.20517350).
- [13] C. Marín, *Unfolding a Graph into a Tree: A Machine-Checked Proof of the Heilmann–Lieb Theorem in Lean 4* (Parte II), 2026. Zenodo, DOI [10.5281/zenodo.20561832](https://doi.org/10.5281/zenodo.20561832).
- [14] C. Marín, *Folding Edges into Vertices: A Machine-Checked Proof of Bass’s Determinant Formula for the Ihara Zeta in Lean 4*, 2026. Zenodo, DOI [10.5281/zenodo.20573120](https://doi.org/10.5281/zenodo.20573120).
- [15] The mathlib Community, *The Lean mathematical library*, en *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs* (CPP 2020), 367–381.